SRM University – AP

Department of Computer Science and Engineering

CHESS GAME IMPLEMENTATION IN C

PROJECT REPORT

TEAM MEMBERS:

G. Vidya Vaishnavi - AP24122230014

P. Manasa - AP24122230016

N. Revanth – AP24122230025

1. Introduction

Chess is a strategic two-player board game played on an 8×8 grid. This project implements a simplified version of chess using the C programming language. The program simulates a chessboard, allows players to input moves using standard algebraic notation (e.g., e2 e4), and validates moves based on basic piece movement rules.

The aim of this project is to demonstrate:

Use of arrays and functions in C

Implementation of game logic

Input validation and control flow

Basic simulation of real-world systems

2. Objectives

To design a console-based chess game in C

To represent a chessboard using a 2D array

To implement movement rules for all chess pieces

To validate user input and enforce turn-based gameplay

To enhance understanding of structured programming concepts

3. System Requirements

Hardware Requirements

Computer with minimum 4GB RAM

Keyboard for input

Software Requirements

C compiler (GCC, Turbo C, etc.)

Operating System: Windows/Linux/Mac

4. Program Overview

The program initializes a chessboard with standard starting positions and allows two players (White and Black) to take turns entering moves.

Key Features

Standard 8×8 chessboard

Turn-based gameplay

Move validation for each piece

Board display after each move

Input using algebraic notation

5. System Requirements

Hardware

- Computer with basic configuration

Software

- C Compiler (GCC)
- Any IDE or text editor

6. Data Structures Used

6.1 Board Representation

char board [SIZE][SIZE];

A 2D array represents the chessboard

Uppercase letters = White pieces

Lowercase letters = Black pieces

' ' represents an empty square

6.2 Piece Representation

Piece	White	Black
Pawn	P	p
Rook	R	r
Knight	N	n
Bishop	B	b
Queen	Q	q
King	K	k

7. Functional Modules

7.1 Board Initialization (in it Board)

Initializes the board with starting positions

Uses memcpy for efficient copying

7.2 Board Display (print Board)

Prints the board with row and column labels

Helps users visualize the game state

7.3 Notation Conversion (notation To Index)

Converts chess notation (e.g., e2) to array indices

Ensures valid input range

7.4 Path Clearance (path Clear)

Checks if path between two positions is clear

Used for rook, bishop, and queen

7.5 Move Validation (valid Move)

Handles rules for each piece:

Pawn: Moves forward one step

Rook: Horizontal/vertical movement

Bishop: Diagonal movement

Queen: Combination of rook and bishop

Knight: L-shaped movement

King: One square in any direction

7.6 Move Execution (make Move)

Validates input and move legality

Updates board if move is valid

Ensures correct player turn

8. Algorithm

Step-by-Step Flow

Initialize board

Display board

Repeat until program ends:

Ask player for move

Convert input to indices

Validate move

Update board

Switch turn

Display updated board

9.Design Approach

The design of the chess game is based on a modular programming approach. The entire system is divided into smaller functional units such as board initialization, move validation, and game execution. Each function is responsible for a specific task, improving code readability and maintainability.

The chessboard is represented using a two-dimensional array, which allows efficient access and modification of elements. The program follows a top-down design strategy, where the main function controls the overall flow, while helper functions handle specific operations like input conversion and move checking.

10. Control Flow Diagram (Description)

Start

Initialize board

Display board

Input move

Validate move

If invalid → show error

If valid → update board

Switch player

Repeat loop

11. Sample Input and Output

Input

White's move: e2 e4

Black's move: e7 e5

Output

```
  a b c d e f g h
8 r n b q k b n r 8
7 p p p p   p p p 7
6               6
5         p     5
4         P     4
3               3
2 P P P P   P P P 2
1 R N B Q K B N R 1
```

12. Error Handling

- Invalid format input
- Moving empty square
- Moving wrong player's piece
- Illegal moves

13. Methodology

The project is divided into modules:

- Board initialization
- Input handling
- Move validation
- Game loop execution

14. Limitations

No check/checkmate detection

No castling

No pawn promotion

No en passant rule

No AI opponent

No move history tracking

15. Advantages

Simple and easy to understand

Demonstrates core C programming concepts

Modular design with functions

Interactive console-based gameplay

16. Disadvantages

- No full chess rules
- No GUI
- No AI opponent

17. Applications

- Learning tool
- Programming practice
- Base for advanced projects

18. Future Enhancements

Add check and checkmate detection

Implement special moves (castling, promotion)

Develop AI opponent

Add GUI using libraries (e.g., SDL or GTK)

Save and load game functionality

19. Conclusion

This project successfully implements a basic version of a chess game using C. It demonstrates how real-world systems can be modeled using programming concepts such as arrays, functions, and control

structures. While simplified, the program provides a strong foundation for developing more advanced chess engines.

20. References

C Programming Language – Kernighan & Ritchie

Online C documentation (GCC)

Chess rules (FIDE official guidelines)

21. Appendix (Code Explanation Summary)

Key Concepts Used:

Arrays (2D)

Functions

Conditional statements

Loops

String handling

Character manipulation (ctype.h)